

FPGA & VHDL

Fiches Astuces / Remarques / Idées — Révision Examen

1. LUT — Look-Up Table

Principe fondamental :

Une LUT ne **calcule pas** une fonction logique — elle **mémore directement le résultat** dans une petite mémoire. Les entrées servent d'**adresse** pour lire la sortie.

Taille d'une LUT à N entrées :

Type de LUT	Entrées	Cases mémoire (bits)	Exemple de fonction
LUT2	2	4	AND, OR, XOR, NAND...
LUT3	3	8	Somme demi-add, retenue FAdder...
LUT4	4	16	Fonctions complexes à 4 variables

Comment lire le bitstream d'une LUT :

Le bitstream d'une LUT = la colonne **Sortie** de la table de vérité, lue de haut en bas (entrées de 00...0 à 11...1).

Exemple XOR (A, B) : table de vérité → 0, 1, 1, 0 → bitstream = **0110**

■ ASTUCES EXAMEN :

✓ 1 LUT par fonction logique élémentaire (1 sortie).

✓ Si une fonction a 2 sorties (ex: HAdder : s et cout), il faut **2 LUTs**.

✓ Le bitstream d'une LUT = colonne sortie de la table de vérité (de haut en bas).

✗ Ne pas confondre bitstream LUT et bitstream FPGA complet.

2. FPGA — Structure et Fonctionnement

Structure interne :

Composant	Rôle
Blocs Logiques (BL)	Contiennent les LUTs + registres (bascules D)
Matrice d'interconnexion	Relie les blocs entre eux (fils programmables)
Pins d'E/S	Communication avec l'extérieur du circuit
Mémoire de configuration	Stocke le bitstream qui configure tout le FPGA

Le Bitstream FPGA complet :

Bitstream FPGA = contenu LUT1 + contenu LUT2 + ... + configuration des interconnexions + configuration des registres. C'est le **fichier binaire total** envoyé au FPGA pour le programmer.

Processus de synthèse :

■ ASTUCES EXAMEN :

- ✓ Un bloc logique contient généralement **2 LUTs**.
- ✓ Compter le nombre de fonctions (sorties) = nombre de LUTs nécessaires.
- ✓ Blocs non utilisés → LUTs remplies de **00000000** (ne font rien).
- ✓ Le FPGA exécute tout en **parallèle** (≠ microprocesseur séquentiel).
- ✗ **Bitstream LUT ≠ Bitstream FPGA complet (le FPGA inclut aussi les connexions).**

3. VHDL — Approche Structurale**Principe de l'approche structurale :**

On décrit le circuit comme un **assemblage de composants** interconnectés (comme un schéma électrique), sans décrire le comportement interne.

Instanciation directe (sans redéclarer le component) :

```
U0 : entity work.NOM_ENTITE port map ( port1 => signal1, port2 => signal2, ... );
```

Astuce clé — Boucle GENERATE :

Pour N composants identiques en cascade :

```
NOM_GEN : for i in N-3 downto 1 generate NOM_inst : entity work.MUX port map ( Sel => Sel(i), Sin0 => S(i), Sin1 => W(i), MuxOut => W(i-1) ); end generate NOM_GEN;
```

■ ASTUCES EXAMEN :

- ✓ Toujours traiter **séparément le 1er et le dernier composant** (cas limites : connexions différentes des cas intermédiaires).
- ✓ Le **generate** s'applique uniquement aux composants intermédiaires (même schéma de connexion répétitif).
- ✓ Déclarer des **signaux internes** (ex: W, C, R0, R1) pour relier les composants.
- ✓ Utiliser **entity work.NOM** pour instancier sans redéclarer le component.
- ✗ **Ne pas oublier les signaux intermédiaires dans la zone signal (avant begin).**

4. Exemple : Additionneur 2 bits (2bitsAdder)**Logique de conception :**

Bit	Composant	Pourquoi ?	Entrées	Sorties
Bit 0 (LSB)	HAdder	Pas de retenue entrante	A(0), B(0)	R0, C
Bit 1 (MSB)	FAdder	Retenue C vient du bit 0	A(1), B(1), C	R1, Cout

Fonctions logiques et leur LUT :

LUT	Fonction	Bitstream
Bloc1 / LUT1	$R0 = A(0) \text{ XOR } B(0)$	0110
Bloc1 / LUT2	$C = A(0) \text{ AND } B(0)$	0001

Bloc2 / LUT1	$R1 = A(1) \text{ XOR } B(1) \text{ XOR } C$	10010110
Bloc2 / LUT2	$Cout = (A(1) \& B(1)) \text{ OR } (C \& (A(1) \text{ XOR } B(1)))$	11101000

■ ASTUCES EXAMEN :

✓ Le **premier bit** d'un additionneur multi-bits utilise toujours un **HAdder** (pas de Cin), les suivants utilisent **FAdder**.

✓ Le **process(clk)** sert à **synchroniser la sortie** (registre sur front montant). Ne pas l'oublier si le port clk existe.

✓ Signaux intermédiaires obligatoires : **C** (retenue), **R0**, **R1** (résultats avant registre).

✗ **Ne pas connecter directement la sortie d'un composant au port de sortie de l'entité si un registre est requis.**

5. Exemple : Système de N MUX en cascade

Structure du système :

N MUX chaînés : chaque MUX sélectionne entre un signal d'entrée Si et la sortie du MUX précédent. Signaux de sélection : Sel(0) à Sel(N-1).

Les 3 cas à distinguer :

Cas	Composant	Sin0 (entrée 0)	Sin1 (entrée 1)	Sortie
1er MUX (N-1)	MUX_N1	S(N-1)	S(N)	W(N-2)
MUX intermédiaires (boucle generate)	MUX_i	S(i)	W(i)	W(i-1)
Dernier MUX (0)	MUX_0	S(0)	W(0)	Sout (port externe)

■ ASTUCES EXAMEN :

✓ **L'astuce structurelle** = instancier explicitement le 1er et le dernier, puis utiliser **generate** pour les intermédiaires.

✓ Les signaux **W(i)** sont des tableaux de std_logic_vector déclarés comme : signal W : array(0 to N-2) of std_logic_vector(2 downto 0);

✓ Vérifier le sens du port map : Sin0 = entrée 0, Sin1 = entrée 1 (conformément à la déclaration de l'entité MUX).

✗ **Le 1er MUX n'a pas de signal W en entrée (ses deux entrées sont externes).**

✗ **Le dernier MUX sort vers Sout (port externe), pas vers un W(i).**

6. Récapitulatif — Méthode à l'examen

Étape	Action
Étape 1	Lire l'entité (entity) : identifier tous les ports (in/out) et leurs types.
Étape 2	Identifier les composants à instancier et leurs connexions.
Étape 3	Déclarer les signaux internes nécessaires (entre composants).

Étape 4	Instancier le 1er composant (cas particulier), puis le dernier.
Étape 5	Utiliser generate pour les composants intermédiaires répétitifs.
Étape 6 (FPGA)	Lister toutes les fonctions logiques → 1 fonction = 1 LUT.
Étape 7 (FPGA)	Construire la table de vérité de chaque LUT → extraire le bitstream.
Étape 8 (FPGA)	Remplir le tableau blocs/LUTs et assembler le bitstream global.